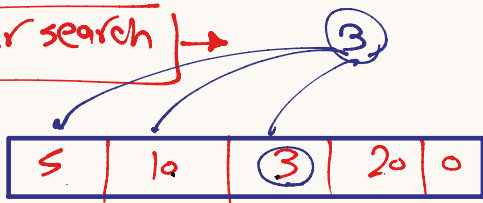


* Algorithm $\rightarrow$ Search/Sort

$\rightarrow$ search $\rightarrow$ coding way to find specific value

- $\rightarrow$ linear search
- $\rightarrow$ Binary search

* linear search



* $\rightarrow$ check the search value with each element in our Array

$\rightarrow$ Adv $\rightarrow$ very simple
 $\rightarrow$ Can work on unordered Array

Note
 ① $\rightarrow$ slow $\rightarrow$ Based on size
 ② Big O $\rightarrow$ Big O $\rightarrow$ describe execution space

disadvantage

$\rightarrow$ very slow with large Array

* ex. write c. code to find index for Input value.

```
#include <stdio.h>
int main()
{
    char no[5] = "5";
    char count = 0;
    for (count = 0; count < 5; count++)
    {
        scanf("%hd", &no[count]);
    }
    printf("Enter search value:");
    char searchvalue = 0;
    scanf("%hd", &searchvalue);
}
```

11 linear

```
for (count = 0; count < 5; count++)
{
    if (no[count] == searchvalue)
    {
        printf("%d", count);
        break;  $\rightarrow$  If keyword to terminate the loop
    }
}
```

* Big O

$\rightarrow$ Mathematical equation
 $\rightarrow$ to describe the
 $\rightarrow$ execution time
 $\rightarrow$ space complexity

$\rightarrow$ time $\rightarrow$

$\rightarrow O(1) \rightarrow$ Constant time
 $\rightarrow O(n \log n) \rightarrow$ logarithmic time
 $\rightarrow O(n^2) \rightarrow$ Quadratic time

* Return 5:10

$\rightarrow$ write C. Code to get the No of Repeat for Input No from user in Input Array

* searching $\rightarrow$ Binary

$\rightarrow$ use to find specific element in ordered array

$\rightarrow O(\log n) \rightarrow$ Based on Array divided

ex. array [5] $\rightarrow$ start

$$\text{Mid} = \frac{\text{start} + \text{End}}{2} = \frac{0 + \text{size} - 1}{2}$$

$$\text{Mid} \Rightarrow \frac{0 + 4}{2} = 2$$

20	[0]
32	[1]
45	[2]
60	[3]
92	[4]

search value == array[Mid]

search value > array[Mid]

search value < array[Mid]

find value

find value example (44)

* start = 0; End = 5;

$$\text{Mid} = \frac{0 + 5}{2} = [2]$$

value == arr[Mid] $\rightarrow$ X

value > arr[Mid] $\rightarrow$ true

* start = Mid + 1
 2 + 1

End = 5; New Mid = $\frac{3 + 5}{2}$
 Mid = (4)

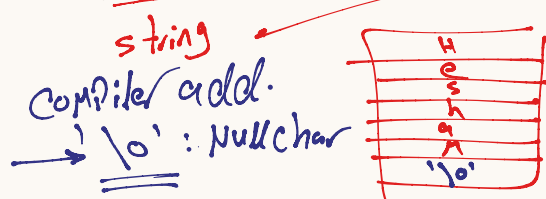
array finished

3	[0]
15	[1]
29	[2]
31	[3]
44	[4]
56	[5]

* string

↳ Array of char

* char name [] = "Hesham";



* size any string = number of char + 1

* Print string

char name [] = "Hesham";
Print f ("%.5", name);

string

* Scan →

char name [20] = "0";
scanf ("%s", name);

Array Name
↓
Address

ex.

short int x = 30; → 2 Bt/e
long int y = 40; → 4 Bt/e
long long int z = 60; → 8 Bt/e

* C version → C89, C90, C99
C11, C17

short int → 2 Bt/e
long → 4 Bt/e
int → 4 Bt/e
long int → 8 Bt/e
long long int → 8 Bt/e

unsigned short int timer = 0; Data type

* typedef → Rename for Data type → Compiler

typedef old Name New Name;

ex. typedef unsigned short int Hamada;

Modifier

* → signed → +ve
-ve
Compiler

Variable
store +ve only
unsigned

* int * char
only

Variable
store +ve & -ve
signed

unsigned char No;
unsigned int No;

signed char
signed int

if you don't select Compiler
will be select int x

char → 1 Bt/e

unsigned char

0000 0000 → 0
1111 1111 → 255

Range No

0 : 255 → +ve

signed char

0000 0000

1111 1111

Range

0 : 127 +ve

-1 : -128 -ve

* Size Modifier

char → at least 1 Bt/e

int → at least 2 Bt/e

↳ 2, 4, 8 Bt/e

Compiler → system Bus

float → at least 4 Bt/e

double → at least 8 Bt/e

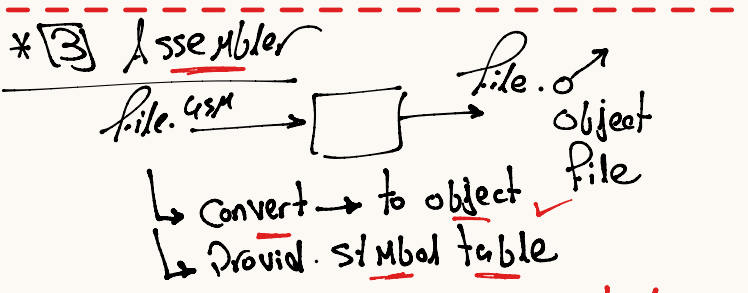
Modifier

① short → int
Compiler → 2 Bt/e
short int

② long → int 4 Bt/e
↳ double

③ long long → int 8 Bt/e

* create Header file
 * std type .h
 typedef unsigned char
 typedef signed char
 unsigned No of bit
 8; No. of bit
 signed



Global Var Function
 x →

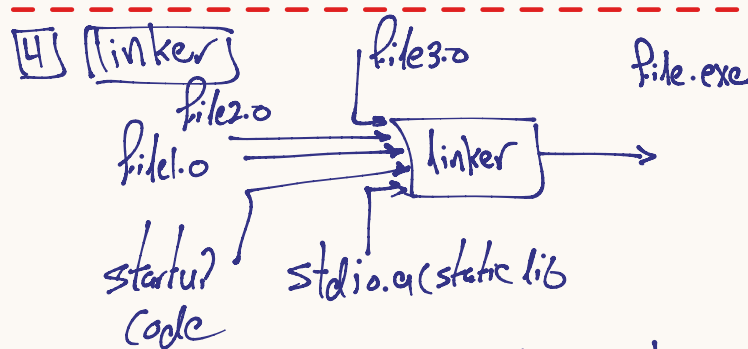
x	provid
printf	need

file.c → file.o

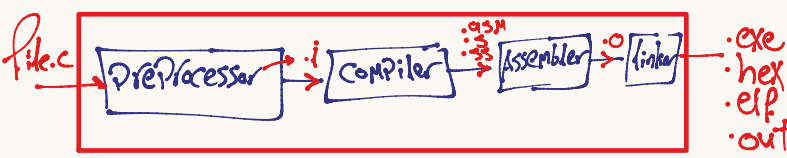
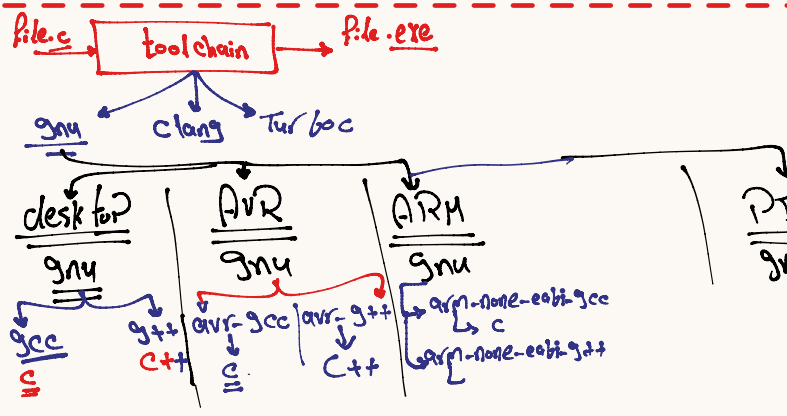
```
#include <stdio.h>
int x = 20;
int main()
{
  printf("x.d\n", Add(5,7));
}
```

Symbol table

S/	St
x	Pro
Main	Pro
Pr	Need
Call	Need



* → linker will be link with different files but symbol table
 * → Collect all object file in one file executable
 ↳ Give Physical Address



[1] PreProcessor

file.c → [] → file.i

* text Replacement for PreProcessor directives.

#include #define #undef
 #ifndef #ifdef #if #else
 #error #warning #elif #endif

gcc -E file.c -o file.i
 ↳ flag → PreProcessor only

[2] Compiler

file.i → [] → file.asm

↳ check syntax error
 ↳ check some logic error
 ↳ optimize
 ↳ Convert to Assembly

gcc -S file.c -o file.asm

* Notes ① #pragma
 ↳ compiler directives
 #pragma once

